Ponteiros

Prof. Thiago Felski Pereira, MSc.

O problema: trocar dois valores

```
void troca(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

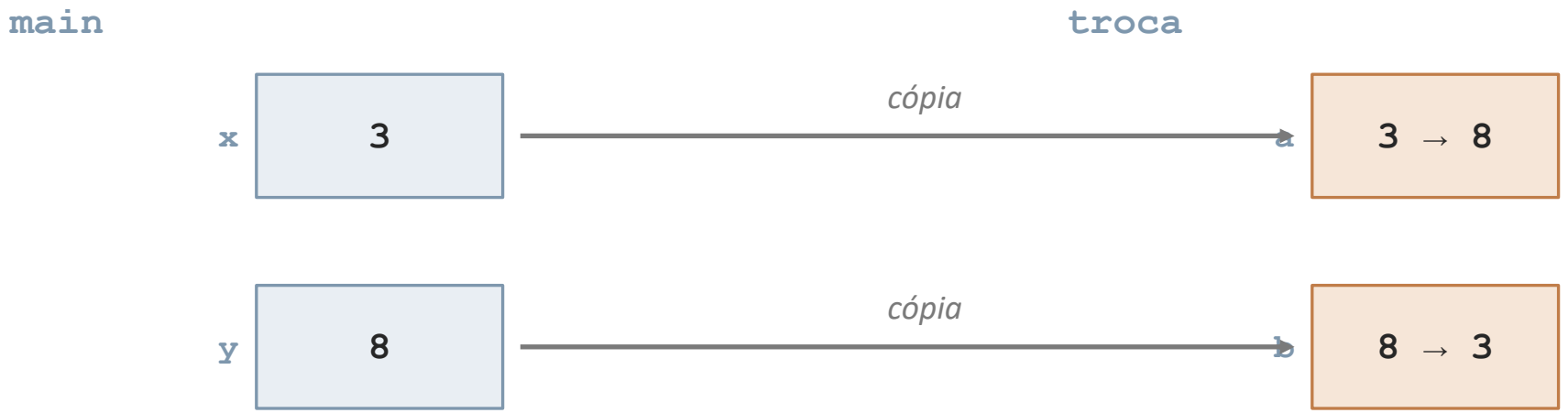
int main() {
    int x = 3, y = 8;
    troca(x, y);
    cout << x << " " << y;
    return 0;
}
```

Pense: o que este programa imprime?

Escreva sua previsão antes de avançar para o próximo slide.

UNIVALI

A função trabalha com cópias



- A saída é **3 8**: a função trocou `a` e `b`, que são cópias.
- `x` e `y` continuam intactos, porque a função nunca soube **onde** eles estão.

UNIVALI

Já resolvemos isso com referência

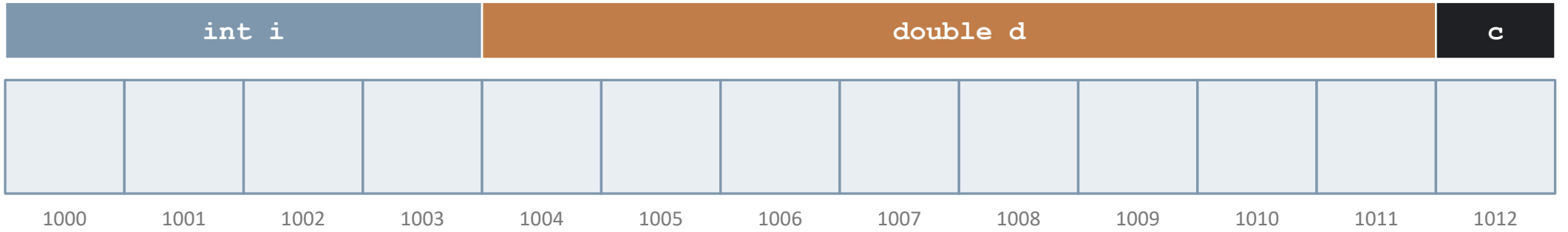
```
void troca(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}

troca(x, y);    // x e y trocam
```

- Com **&**, a função recebe a **própria variável**, não uma cópia.
- Para isso ela precisa saber **onde a variável está na memória**.
- Esse "onde" é o **endereço**. Hoje vamos trabalhar com endereços de forma explícita.

UNIVALI

Memória: uma rua com casas numeradas



- A RAM é uma sequência de bytes. A posição de cada byte é o seu **endereço**.
- Se `e` é o endereço de um byte, `e+1` é o endereço do byte seguinte.
- Uma variável ocupa bytes consecutivos. Seu endereço é o do **primeiro byte**.

UNIVALI

Quantos bytes cada tipo ocupa?

Tipo	Bytes	Observação
`char`	1	sempre 1
`bool`	1	
`int`	4	na maioria dos computadores
`float`	4	
`double`	8	
ponteiro	8	em sistemas de 64 bits

```
cout << sizeof(int);
cout << sizeof(double);
```

Saída:

```
4
8
```

- O operador `sizeof` informa o número de bytes.

UNIVALI

Medindo variáveis

PRÁTICA 1

Antes de executar: qual valor você espera que o programa imprima?

```
struct DATA {
    int dia, mes, ano;
};

int main() {
    cout << sizeof(DATA);
    return 0;
}
```

- Compile, execute e compare com a sua previsão.
- Acrescente um campo `double nota;`. O que muda?
- Teste `sizeof(string)` e compare com o colega.

UNIVALI

O operador &: endereço de uma variável

```
int i = 1234;
cout << i << "\n";
cout << &i << "\n";
```

Saída (o endereço muda a cada execução):

```
1234
0x7ffe5c2a3b24
```

- `&i` lê-se "endereço de `i`".
- O endereço aparece em **hexadecimal** (0x...).
- Nos slides usaremos números simples (1000, 1004...) para facilitar o desenho.

UNIVALI

Endereços de um vetor

PRÁTICA 2

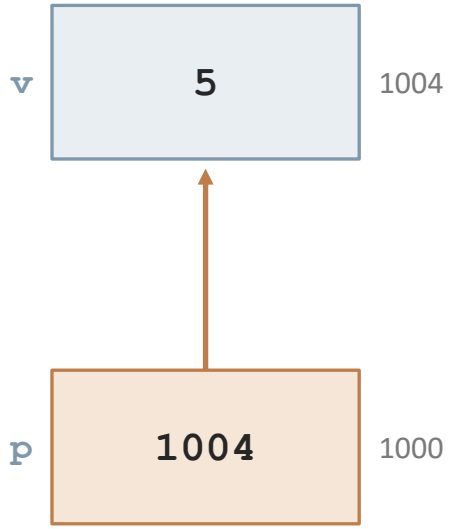
```
int v[4] = {10, 20, 30, 40};
for (int i = 0; i < 4; i++)
    cout << &v[i] << "\n";
```

- Compile e execute.
- Qual é a diferença (em bytes) entre dois endereços seguidos?
- Por que esse valor?
- Troque `int` por `double`. O que muda?

UNIVALI

Ponteiro: uma variável que guarda um endereço

```
int v = 5;
int *p = &v;
```



- `p` guarda o endereço de `v`: dizemos que **p aponta para v**.
- `*p` é o valor que está no endereço guardado em `p`. Aqui, `*p` vale 5.
- `int *` lê-se "ponteiro para int".

UNIVALI

Os símbolos & e *

Símbolo	Onde	Significa	Exemplo
`&`	antes de uma variável	endereço de	`&v`
`*`	na declaração	p é ponteiro para o tipo	`int *p;`
`*`	em uma expressão	conteúdo apontado por p	`cout << *p;`

Armadilha: o `*` pertence a cada variável, não ao tipo.

```
int *a, b;     // a é ponteiro; b é um int comum
int *c, *d;    // os dois são ponteiros
```


UNIVALI

Exemplo passo a passo

```
int v = 5;
int *p = &v;
cout << *p;     // 5
cout << p;      // endereço de v
*p = 9;
cout << v;      // 9
```

- `*p = 9` altera o valor de `v`.
- Não existe cópia: `p` leva até a variável original.
- Alterar `*p` e alterar `v` são a mesma coisa.

UNIVALI

Rastreie as caixas

PRÁTICA 3

```
int *p1, *p2, v = 5, x = 3;
p1 = &v;
p2 = &x;
*p1 = *p2;
cout << *p1 << " " << *p2 << "\n";
*p1 = 9;
cout << *p1 << " " << *p2 << "\n";
p2 = p1;
*p2 = 10;
cout << *p1 << " " << *p2 << " " << v << " " << x;
```

- Desenhe as caixas **v**, **x**, **p1** e **p2**.
- Preveja as **três** linhas de saída.
- Depois execute e confira.

UNIVALI

Ponteiro sem destino: cuidado

```
int *p;          // guarda um valor qualquer
*p = 7;          // ERRO: escreve em local desconhecido
```

```
int *q = nullptr;      // aponta para "lugar nenhum"
if (q != nullptr) {
    cout << *q;        // só usa se for válido
}
```

- Um ponteiro não inicializado guarda lixo.
- Inicialize sempre: com um endereço válido ou com **`nullptr`**.
- Antes de usar **`*q`**, teste se **`q != nullptr`**.

UNIVALI

Por que está errado?

PRÁTICA 4

```
void troca(int *i, int *j) {
    int *temp;
    *temp = *i;
    *i = *j;
    *j = *temp;
}
```

- Para onde `temp` aponta?
- O que acontece na linha `*temp = *i;`?
- Corrija a função.

UNIVALI

Ponteiros em funções: a troca

```
void troca(int *i, int *j) {
    int temp = *i;
    *i = *j;
    *j = temp;
}

int main() {
    int x = 3, y = 8;
    troca(&x, &y);
    cout << x << " " << y;
    return 0;
}
```

Saída:

```
8 3
```

- `&x` entrega à função o **endereço** de `x`.
- Dentro dela, `*i` **é** a variável `x`.

UNIVALI

Referência ou ponteiro?

	Referência (&)	Ponteiro (*)
Chamada	`troca(x, y)`	`troca(&x, &y)`
Dentro da função	`a`	`*a`
Pode ser "nulo"?	não	sim (`nullptr`)
Pode mudar de alvo?	não	sim

- Se basta alterar uma variável, a referência é mais simples e segura.
- Ponteiros são necessários quando o endereço precisa ser **guardado, somado ou testado**: vetores, alocação dinâmica, estruturas ligadas.

UNIVALI

Saídas por ponteiro: horas e minutos

PRÁTICA 5

Escreva a função **hm**, que converte minutos em horas e minutos. Ela recebe `mnts` e os **endereços** de duas variáveis, onde deposita o resultado.

```
void hm(int mnts, int *h, int *m);

int main() {
    int h, m;
    hm(135, &h, &m);
    cout << h << "h" << m << "min";    // 2h15min
}
```


UNIVALI

Saídas por ponteiro: menor e maior

PRÁTICA 6

Escreva a função **mm**, que recebe um vetor e os endereços de duas variáveis, e deposita nelas o **menor** e o **maior** elemento.

```
void mm(int v[], int n, int *min, int *max);

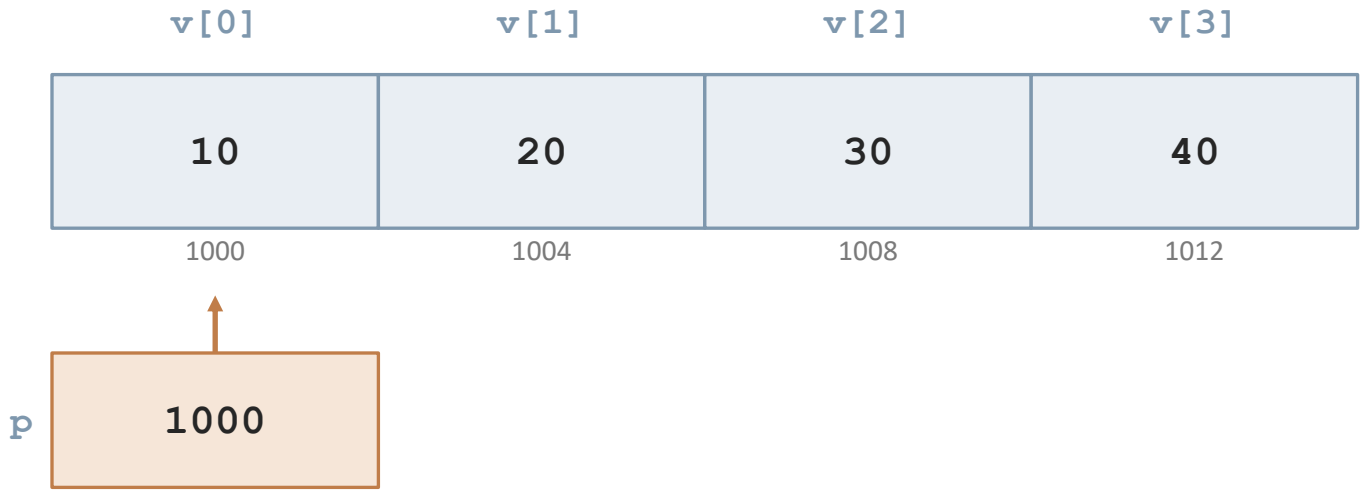
int main() {
    int v[5] = {4, 9, 2, 7, 5};
    int menor, maior;
    mm(v, 5, &menor, &maior);
    cout << menor << " " << maior;    // 2 9
}
```


UNIVALI

Vetores e ponteiros

```
int v[4] = {10, 20, 30, 40};
int *p = v;        // o mesmo que &v[0]
cout << *p;        // 10
```

- O nome do vetor é o endereço do primeiro elemento: `v` é o mesmo que `&v[0]`.
- O vetor **não** é uma variável ponteiro: não dá para fazer `v = outro`.



UNIVALI

Aritmética de endereços

Expressão	Endereço	Conteúdo (*)
`p` ou `v`	1000	10
`p + 1`	1004	20
`p + 2`	1008	30
`p + 3`	1012	40

- Somar 1 a um ponteiro avança **um elemento**, não um byte.
- O compilador multiplica pelo tamanho do tipo: `int` avança 4 bytes, `double` avança 8.
- `v + i` é o endereço de `v[i]`.

UNIVALI

Três formas de percorrer o vetor

```
for (int i = 0; i < 4; i++)
    cout << v[i] << " ";

for (int i = 0; i < 4; i++)
    cout << *(v + i) << " ";

for (int *c = v; c < v + 4; c++)
    cout << *c << " ";
```

Cada laço imprime:

```
10 20 30 40
```

- `v[i]` é o mesmo que `*(v + i)`.
- `&v[i]` é o mesmo que `v + i`.

UNIVALI

Aritmética de endereços

PRÁTICA 7

- Os elementos de `v` são `int` de 4 bytes e o endereço de `v[0]` é **55000**. Qual é o valor da expressão `v + 3`?
- Se `i` é inteiro e `v` é vetor de inteiros, descreva em português a sequência de operações para calcular `&v[i + 9]`.
- Qual é a diferença conceitual entre `v[3]` e `v + 3`?

UNIVALI

Vetor como parâmetro é um ponteiro

```
void zera(int v[], int n) {     // v é, na verdade, int *v
    for (int i = 0; i < n; i++)
        v[i] = 0;
}
```

- Por isso a função altera o vetor original: nunca houve cópia, só o endereço do primeiro elemento.
- Dentro da função, **`sizeof(v)`** devolve o tamanho de um ponteiro (8), e não o do vetor. Por isso passamos **`n`**.

UNIVALI

O que faz esta função?

PRÁTICA 8

```
void imprime(char *v, int n) {
    char *c;
    for (c = v; c < v + n; c++)
        cout << *c;
}
```

- O que a função faz?
- Reescreva usando `v[i]`.
- Chame-a com `char s[4] = {'a', 'b', 'c', 'd'};` e confira.

UNIVALI

Ponteiros e structs: o operador ->

```
struct Aluno {
    string nome;
    float nota;
    bool aprovado;
};

void avalia(Aluno *a) {
    a->aprovado = (a->nota >= 6);
}

Aluno ana = {"Ana", 7.5, false};
avalia(&ana);
```

- Com ponteiro para struct, o campo é acessado pela seta `->`.
- `a->nota` é o mesmo que `(*a).nota`.
- Regra: `.` com variável, `->` com ponteiro.

UNIVALI

Reajuste de nota

PRÁTICA 9

Usando a struct `Aluno` do slide anterior, escreva a função **reajusta**, que soma `pontos` à nota do aluno (limitada a 10) e atualiza o campo `aprovado` (nota >= 6).

```
void reajusta(Aluno *a, float pontos);

reajusta(&ana, 1.0);
```

- Teste com um aluno em 5.5 e outro em 9.5.

UNIVALI

Resumo

Símbolo	Lê-se	Exemplo
`&v`	endereço de v	`int *p = &v;`
`int *p`	p é ponteiro para int	`int *p = nullptr;`
`*p`	conteúdo apontado por p	`*p = 9;`
`p->campo`	campo da struct apontada	`a->nota`
`p + i`	i elementos adiante	`*(p + 2)`
`nullptr`	ponteiro sem destino	`if (p != nullptr)`

Lembre: inicialize todo ponteiro e nunca use `*p` sem ter certeza de que `p` aponta para algo válido.

UNIVALI

E se não soubermos o tamanho do vetor?

```
int n;
cin >> n;
int v[n];      // não é C++ padrão
```

- Tamanho de vetor deve ser conhecido ao compilar.
- Com ponteiros podemos pedir memória **durante a execução**.
- Próxima apresentação: alocação dinâmica (`new` e `delete`).

UNIVALI